MEIKURAL Version 1
Tonic-aligned STFT images and a small CNN

Implementation and mathematical reference

Contents

1 Purpose and experimental contract **1**

2 Configuration, commands, and artifacts **1**
2.1 Installation 1
2.2 Execution 2

3 Decoding, windowing, and amplitude **2**
3.1 Bounded decoding 2
3.2 Sample-index window definition 2
3.3 Gain normalization 3

4 Audio-only tonic estimation **3**

5 STFT and the CNN image **4**
5.1 Time–frequency analysis 4
5.2 Tonic-relative frequency resampling 4
5.3 Log compression and storage 4

6 Feature cache and failure traceability **5**

7 CNN mathematics and dimensions **5**
7.1 Receptive fields 5

8 Generated numerical reference **6**
8.1 Configured settings 6
8.2 Derived CNN dimensions 7

9 Training objective, optimizer, and checkpoints **7**

10 Devices, inference aggregation, and interpretation **8**
10.1 Interactive single-passage inference 8
10.2 Folder-labelled batch reports 8

11 Verification and experiment status **9**

1 Purpose and experimental contract

Version 1 maps raw audio to ranked scores for seven Hindustani raags. Every feature is computed from audio alone. A tonic-relative spectrogram retains harmonics, accompaniment, and melodic movement; a two-dimensional CNN learns local patterns in that matrix. No predominant-melody extractor is required.

All eligible music in `data/ver1/set` is training data. There is no internal validation or test split. External validation and final test audio will be supplied later; neither is invented or inferred from the current recordings. Training accuracy is not an estimate of generalization. Popular songs can mix raags, change tonic, or fall outside the supported classes.

Raag	Source recordings
Shree	5
Jog	3
Miyan Malhar	3
Lalit	3
Todi	3
Marwa	3
Bhairavi	5
Total	25

Class order is stored in the JSON configuration and copied into every checkpoint; folder spelling supplies labels. Whole recording folders and annotations remain unchanged. Saraga's supplied tonic is used only for optional diagnostics, never for features, fallback, or classification.

2 Configuration, commands, and artifacts

`src/ver1/config.json` is the single editable source of operational and mathematical settings. Relative dataset, feature, and model paths resolve against the repository, independently of the shell's working directory. Commands below run from the repository root. Change output paths when starting a different experiment; existing checkpoints are protected from accidental overwrite.

2.1 Installation

Use the project Python 3.12 virtual environment. Install all project, notebook, and development dependencies from the single requirements file:

```
.venv/bin/python -m pip install -r requirements.txt
```

Project dependencies are pinned; pip resolves their transitive dependencies. The unified requirements select `torch==2.9.0+cu126` from the CUDA 12.6 index, replacing any previously installed CPU-only build. To update only Torch:

```
.venv/bin/python -m pip install torch==2.9.0+cu126 \
   --index-url https://download.pytorch.org/whl/cu126
```

Training defaults to explicit CUDA and fails if CUDA is unavailable. CPU mode and automatic fallback remain available through explicit configuration changes. No torchvision or torchaudio dependency is required. FFmpeg and FFprobe must be on PATH for codecs not handled by libsndfile. Essentia's Python wheel is pinned to 2.1b6.dev1389; its upstream license is AGPL-3.0-only.

2.2 Execution

```
.venv/bin/python -m src.ver1.preprocessing.prepare
.venv/bin/python -m src.ver1.training.train
.venv/bin/python -m src.ver1.inference.predict \
   /path/to/audio.wav models/ver1/latest.pt
# Optional audio-only tonic comparison with supplied annotations:
.venv/bin/python -m src.ver1.preprocessing.prepare --diagnose-tonic
# Regenerate numerical documentation after changing the configuration:
```

```
.venv/bin/python -m src.ver1.documentation.generate
mkdir -p outputs/ver1/docs
cd docs
pdflatex -interaction=nonstopmode -halt-on-error \
    -output-directory=../outputs/ver1/docs ver1.tex
```

Preparation and training accept `-config PATH`; documentation generation also accepts `-config` and `-output`. Inference takes an audio path and checkpoint path, plus optional `-device auto|cpu|cuda`.

The Python interface is `predict(audio_path, checkpoint_path, device="auto")`. It returns audio duration, device, class order, window metadata and failures, per-window scores, and an overall ranking. The notebook `notebooks/02_ver1_inference.ipynb` imports this function; the user normally edits only the audio and checkpoint paths. It shows rankings, tonics, padding, failures, and a timestamped score timeline. No trained checkpoint is distributed by the implementation smoke test.

3 Decoding, windowing, and amplitude

3.1 Bounded decoding

Native libsndfile blocks are converted to mono using the arithmetic channel mean and resampled by a stateful soxr HQ stream. Maintaining resampler state across blocks avoids artificial boundaries. Unsupported codecs use FFmpeg with an explicit arithmetic downmix and conversion to mono float32 at the feature sample rate. Source audio is never rewritten. The decoder, sliding buffer, and inference batches are bounded in size; neither training nor inference loads an entire long recording into memory. Result metadata grows with the number of windows. Missing, empty, corrupt, or nonfinite audio produces an error, not a prediction from a knowingly truncated file.

3.2 Sample-index window definition

Let f_s be the feature sample rate, L the window length in samples, S the stride, and a_0 the first start. The configuration must map all three times to whole sample counts. For decoded length $N > 0$:

$$a_j = a_0 + jS, \qquad J = \max\left(0, \left\lceil \frac{N - L - a_0}{S} \right\rceil\right).$$

Generate $j = 0, \ldots, J$ only. The last window is the first to reach or cross EOF. For each interval $[a_j, a_j + L)$, retain real samples on $[\max(0, a_j), \min(N, a_j + L))$, with

$$p_{\text{left}} = \max(0, -a_j), \qquad p_{\text{right}} = \max(0, a_j + L - N).$$

The valid length plus both paddings equals L. Exact-boundary windows get no additional trailing window. A one-second input produces one window with 15 seconds left padding and 14 seconds right padding under the defaults. A 70-second input gives starts $-15, 0, 15, 30, 45$ seconds and a final five-second right pad. The 15-second clip stride differs from the much shorter STFT hop.

3.3 Gain normalization

For real, unpadded samples $x_1, \ldots, x_n$, compute

$$r = \sqrt{\frac{1}{n}\sum_i x_i^2}, \quad p = \max_i |x_i|, \quad g = \min\left(\frac{10^{R/20}}{r}, 10^{G/20}, \frac{P}{p}\right).$$

Here R is the target dBFS RMS, G the maximum amplification in dB, and P the peak ceiling. Reject a window if r is below the configured silence threshold; otherwise apply gx and then add zeros. RMS accumulation uses float64; normalized audio and features use float32. No hard clipping

is used. The target may be unreachable under peak or gain limits. Equal RMS is equal energy, not a guarantee of equal perceived loudness. Per-matrix peak-relative dB compression later largely cancels scalar gain; normalization still controls tonic-estimator input level and protects headroom.

4 Audio-only tonic estimation

Sa is the reference frequency against which melodic intervals are measured. Estimate it independently for each window, using only normalized real audio resampled to the estimator sample rate. Artificial padding is excluded. The same information is available for a standalone external clip.

Essentia's `TonicIndianArtMusic` computes a multipitch harmonic salience representation, accumulates pitch evidence in a histogram, and selects tonic using relationships between prominent peaks and its built-in decision tree [1, 2]. It does not train on our raag labels. The configured frame/hop, cents resolution, harmonic count/decay, compression, threshold, salience peaks, reference frequency, and search interval are all listed in the generated tables. The estimator reference frequency is an internal coordinate origin, not a supplied tonic. The search range constrains possible tonic values.

No-peak runtime failures, nonfinite estimates, and out-of-range tonics mark the window unusable. Missing dependencies or invalid algorithm construction fail the run immediately. No guessed tonic or annotation fallback is provided. The algorithm supplies a tonic frequency, not a calibrated confidence score; an accepted result can still be musically incorrect.

Optional diagnostics read the recording's unique `.ctonic.txt` file only after extraction. For annotation f_0 and estimate $\widehat{f}_0$, report

$$e = 1200 \log_2(\widehat{f}_0/f_0), \quad q = \mathrm{round}(e/1200), \quad e_{\mathrm{corrected}} = e - 1200q.$$

q indicates the nearest octave displacement; it is not used to repair features. Recording annotations are a diagnostic reference rather than proof that tonic is constant or perfectly annotated throughout each recording.

5 STFT and the CNN image

5.1 Time–frequency analysis

For normalized padded audio y, analysis length W, FFT size F, hop H, and periodic Hann window w, the default centered STFT is

$$Z[k,t] = \sum_{n=0}^{F-1} \widetilde{y}[tH + n - F/2]\, \widetilde{w}[n] e^{-2\pi i k n/F}, \qquad M[k,t] = |Z[k,t]|.$$

The analysis window is centered and zero-extended to the FFT size when $W < F$; audio outside the clip is zero. Defaults use $W = F$, even F, and retain $k = 0, \ldots, F/2$. Librosa's unnormalized forward STFT convention is used [3]. Phase is discarded; the CNN receives magnitude-derived features.

$$f_k = k f_s/F, \qquad t_t = tH/f_s, \qquad T = 1 + \left\lfloor \frac{L + 2\lfloor F/2 \rfloor - F}{H} \right\rfloor.$$

Without frame centering, remove the $2\lfloor F/2 \rfloor$ padding term. Defaults give $T = 1292$, FFT spacing about 2.692 Hz, an analysis duration of about 371.5 ms, and a hop of about 23.22 ms. FFT spacing and the Hann main lobe determine effective frequency discrimination; a dense remapped grid cannot recover detail absent from the STFT. Time localization also reflects the analysis window, not just the hop.

5.2 Tonic-relative frequency resampling

Relative pitch in cents is

$$c = 1200 \log_2(f/\widehat{f}_{\mathrm{Sa}}), \qquad f(c) = \widehat{f}_{\mathrm{Sa}} 2^{c/1200}.$$

Use the configured half-open cents grid $c_i = c_{\min} + i\Delta c < c_{\max}$. For default $c_{\min} = -1200$, $c_{\max} = 4800$, and $\Delta c = 25$, there are 240 rows, ending at 4775 cents. Separate octaves remain separate rows; pitches are not rounded to twelve note classes.

At each requested frequency, let $u_i = f(c_i)F/f_s$, $k_i = \lfloor u_i \rfloor$, and $\alpha_i = u_i - k_i$. Interpolate magnitudes, not plot labels:

$$A[i,t] = (1-\alpha_i)M[k_i,t] + \alpha_i M[k_i+1,t].$$

Validation requires the entire grid to remain below Nyquist at the largest allowed tonic. Matching relative intervals now occupy matching rows across different tonics. Harmonics and accompaniment are retained; remapping is not source separation and does not eliminate differences in timbre or instrumentation.

5.3 Log compression and storage

For $m = \max A$, numerical floor ϵ, and dynamic range B dB: Set $X = 0$ if $m \le \epsilon$. Otherwise:

$$D[i,t] = 20 \log_{10} \frac{\max(A[i,t], \epsilon)}{m},$$
$$X[i,t] = \frac{\mathrm{clip}(D[i,t], -B, 0) + B}{B}.$$

Defaults yield a 240×1292 matrix with values in $[0,1]$. No colors, axes, image resizing, or image-file compression enter the model. Cache each matrix as float16 NumPy with pickles disabled; training converts it to float32. Inference deliberately round-trips float16 to match this quantization.

6 Feature cache and failure traceability

The manifest records the absolute dataset root, sorted source identities, source sizes and modification timestamps, configuration, feature fingerprint, completion status, and every generated window. Window entries contain class, recording ID, start, valid interval, left/right padding, estimated tonic, feature filename, and failure reason. Near-silence and tonic failures appear as entries without features. Every class must retain usable examples before training.

The fingerprint is SHA-256 of canonical JSON containing an explicit format version and the audio, tonic, and spectrogram sections. Changing an algorithm's feature semantics requires bumping this format version. It is independent of optimizer settings. A completed matching cache is reused after checking feature files exist. Source identity or preprocessing mismatch refuses reuse and asks for a new cache directory. Source timestamps/sizes are practical change detection, not a cryptographic content audit. Package versions are recorded in model checkpoints.

Features and manifests are published using temporary files and rename. An interrupted incomplete preparation is rebuilt on the next matching run; it does not silently become training-ready. The source and cache directories must be separate and non-nested. Decode failures stop preparation. No intermediate audio files are required. The `data/` directory and unselected model runs remain ignored. Git explicitly permits the small trained `models/ver1/latest.pt` checkpoint and its config/history JSON files for laptop inference. They can be committed without the training dataset or feature cache. The checkpoint loads on CPU even when trained on CUDA; inference device selection is independent of the saved training device. The README documents installation on a Linux CPU laptop using the unified dependencies with the workstation Torch pin excluded.

7 CNN mathematics and dimensions

Input batches have shape $(B, 1, 240, 1292)$. Every block applies biased stride-one same-padded cross-correlation, ReLU, and floor-mode max pooling:

$$z_{b,o,h,t} = b_o + \sum_{i,u,v} K_{o,i,u,v} X_{b,i,h+u-p_h,t+v-p_t}, \qquad a = \max(0, z).$$

Kernel dimensions are odd, with $p_h = (k_h - 1)/2$ and $p_t = (k_t - 1)/2$. Max pooling uses kernel and stride $(2, 4)$ by default. For channel count C_o, incoming channels C_i, and kernel (k_h, k_t), parameters are $C_o(C_i k_h k_t + 1)$. Pooling, ReLU, flattening, and dropout have no parameters.

Adaptive average pooling preserves 12 pitch regions and averages time down to one column. For input length I and output length O, output bin j averages indices from $\lfloor jI/O \rfloor$ through $\lceil (j+1)I/O \rceil - 1$; nondivisible bins may overlap. Flattening produces 768 values. Training-only dropout independently zeros entries with probability 0.25 and scales retained entries by $1/(1-0.25)$. A biased linear layer maps to seven logits.

Default convolution parameter counts are 352, 12832, and 18496. The classifier adds $768 \cdot 7 + 7 = 5383$, giving 37063 total. The next section generates all exact shapes and counts from actual PyTorch modules. If configuration changes alter them, regenerate the tables.

7.1 Receptive fields

For each spatial axis, initialize receptive field $r_0 = 1$ and jump $j_0 = 1$. For kernel k_ℓ and stride s_ℓ:

$$r_\ell = r_{\ell-1} + (k_\ell - 1) j_{\ell-1}, \qquad j_\ell = j_{\ell-1} s_\ell.$$

The generated table gives convolution and max-pool receptive fields in image bins/frames. After the third max pool these are 26×118 with jumps 8×64. An interior 118-frame time receptive field covers roughly $(117H + W)/f_s = 3.09$ seconds of audio. Adaptive pooling then aggregates across positions. This simple CNN models local patterns and their regional averages; it does not explicitly model the order of an entire 30-second melodic phrase.

8 Generated numerical reference

8.1 Configured settings

Setting	Value
paths.dataset	"data/ver1/set"
paths.features	"data/ver1/features"
paths.run	"models/ver1"
classes	Shree, Jog, Miyan Malhar, Lalit, Todi, Marwa, Bhairavi
audio.sample_rate	22050
audio.window_seconds	30
audio.stride_seconds	15
audio.first_start_seconds	-15
audio.rms_dbfs	-20
audio.max_gain_db	20
audio.peak_ceiling	0.99
audio.silence_rms	1e-05
audio.decode_block_samples	65536
tonic.sampleRate	44100
tonic.minTonicFrequency	65

Setting	Value
tonic.maxTonicFrequency	500
tonic.frameSize	2048
tonic.hopSize	512
tonic.binResolution	10
tonic.numberHarmonics	20
tonic.harmonicWeight	0.85
tonic.magnitudeCompression	1
tonic.magnitudeThreshold	40
tonic.numberSaliencePeaks	5
tonic.referenceFrequency	55
spectrogram.n_fft	8192
spectrogram.win_length	8192
spectrogram.hop_length	512
spectrogram.window	"hann"
spectrogram.center	true
spectrogram.min_cents	-1200
spectrogram.max_cents_exclusive	4800
spectrogram.step_cents	25
spectrogram.dynamic_range_db	80
spectrogram.epsilon	1e-12
model.channels	[16, 32, 64]
model.kernels	[[3, 7], [5, 5], [3, 3]]
model.pool	[2, 4]
model.adaptive_pool	[12, 1]
model.dropout	0.25
training.device	"cuda"
training.epochs	30
training.batch_size	8
training.learning_rate	0.001
training.weight_decay	0.0001
training.betas	[0.9, 0.999]
training.epsilon	1e-08
training.seed	42
training.workers	0
inference.batch_size	8

8.2 Derived CNN dimensions

Feature shape: (240, 1292). Trainable parameters: 37,063.

Layer	Output (C,H,W)	Parameters	RF (H,W)
Input	1 x 240 x 1292	0	1 x 1
Conv2d 1	16 x 240 x 1292	352	3 x 7
MaxPool2d 1	16 x 120 x 323	0	4 x 10
Conv2d 2	32 x 120 x 323	12,832	12 x 26
MaxPool2d 2	32 x 60 x 80	0	14 x 38
Conv2d 3	64 x 60 x 80	18,496	22 x 70
MaxPool2d 3	64 x 30 x 20	0	26 x 118
AdaptiveAvgPool2d	64 x 12 x 1	0	aggregated
Flatten / Dropout	768	0	aggregated
Linear	7	5,383	aggregated

9 Training objective, optimizer, and checkpoints

Let n_c be usable cached windows of class c, $N = \sum_c n_c$, and $C = 7$. Class weights are $w_c = N/(Cn_c)$. For a batch of labels y_b and logits z_b:

$$p_{b,c} = \frac{e^{z_{b,c}}}{\sum_j e^{z_{b,j}}}, \qquad \mathcal{L} = -\frac{\sum_b w_{y_b} \log p_{b,y_b}}{\sum_b w_{y_b}}.$$

Use numerically stable PyTorch cross-entropy on logits, not manually computed softmax. Weighting compensates class window counts, not artist diversity. Epoch loss aggregates batch weighted numerators and denominators; accuracy is ordinary correct predictions divided by windows seen, measured during training with dropout active.

AdamW uses gradients g_t, moments

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$

bias corrections $\widehat{m}_t = m_t/(1-\beta_1^t)$ and $\widehat{v}_t = v_t/(1-\beta_2^t)$, and update

$$\theta_t = (1-\eta\lambda)\theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \epsilon_{\text{Adam}}}.$$

All trainable parameters, including biases, receive the configured decay. Defaults are 30 epochs, batch 8, learning rate 10^{-3}, decay 10^{-4}, $(\beta_1, \beta_2) = (0.9, 0.999)$, and epsilon 10^{-8}. Use shuffled batches, float32, no augmentation, no scheduler, and no early stopping. Seed Python, NumPy, and Torch with the configured seed. Exact repeatability across devices, software builds, or nondeterministic GPU kernels is not guaranteed.

After each epoch, atomically save `latest.pt` and update `history.json`. The checkpoint includes weights, optimizer state, epoch, ordered classes, complete config, feature fingerprint, and package versions. No "best" model is selected from training metrics; automatic resume is not implemented. Choose a new run directory for a new training run. Inference validates the saved config, restores its architecture and preprocessing, loads weights with PyTorch's restricted weights-only loader, and switches to evaluation mode. A current JSON edit cannot silently change an existing checkpoint's model.

10 Devices, inference aggregation, and interpretation

The default training device is `cuda`, which fails clearly if unavailable. With optional device `auto`, choose CUDA only if PyTorch reports it available; otherwise use CPU. Explicit `cpu` always uses CPU. Feature preparation remains on CPU. A CUDA checkpoint can load on CPU. Runtime failures and GPU out-of-memory errors propagate; training is not silently restarted.

The same raw-audio iterator supplies preparation and inference. Inference stacks only a bounded batch of successful matrices; every failed window remains in the result metadata. Softmax supplies seven relative class scores. For successful windows with real durations d_j and score vectors p_j, the overall score is

$$\overline{p}_c = \frac{\sum_j d_j p_{j,c}}{\sum_j d_j}.$$

Duration weighting reduces padded-edge influence but does not remove overlap correlation or give every source sample equal total weight. If all windows fail, return an empty ranking with the reasons rather than guessing. Score timestamps use actual valid intervals; the plot places each score at the real interval's midpoint. Long-audio results should be inspected over time, especially when musical content changes.

No unknown rejection or score calibration is claimed. All successful inputs receive scores among these seven classes. Short clips can be ambiguous; tonic octave errors shift features by 1200 cents, and a successful estimator can still be wrong. Accompaniment, artist identity, and recording conditions can become CNN shortcuts. Many overlapping windows do not create new independent recordings.

10.1 Interactive single-passage inference

`notebooks/03_ver1_interactive_clip.ipynb` offers a start-time slider, an exact-seconds field, audio playback, and an explicit prediction button. The widget dependency is included in the unified requirements. Loading a song decodes it once at the checkpoint sample rate and caches mono float32 audio in RAM; this trades memory for responsive seeking. Very long recordings should use a disk-backed approach in a future extension. The model and tonic estimator are also reused between selections.

For selected start $s \geq 0$, compute $a = \text{round}(sf_s)$, take samples $x[a : \min(a + L, N)]$, and right-pad to L if necessary. No left-padding window or whole-song aggregation is generated. The slider allows complete windows when the song is at least L/f_s seconds long; shorter songs use start zero and right padding. Playback contains only the real selected samples. Normalization, tonic estimation excluding padding, STFT remapping, and float16 quantization match training. A single model forward pass yields the seven softmax scores.

Changing the selection clears stale output without running inference. Changing either path invalidates the loaded state and requires another load. Errors and unusable passages are shown beside the controls. Selecting a vocal passage is not source separation: accompaniment is retained and tonic errors remain possible. Interactive choices used to guide changes are development data, not an independent final test. A static HTML export cannot execute the Python prediction pipeline.

10.2 Folder-labelled batch reports

`notebooks/04_ver1_handpicked_batch.ipynb` applies the existing whole-audio prediction function independently to every supported audio file under the test collection. The first directory component below the collection root supplies its expected label. Case and word separators are normalized against saved model classes; the explicit spelling alias `sree` maps to Shree. Unknown labels are preserved, especially Bhairav versus Bhairavi. Folder labels are not inputs to inference and are assumed rather than independently validated ground truth.

Each performance produces a single Matplotlib figure containing its relative filename, expected and predicted raags, match status, window-score timeline, and ranked overall scores. The ranking is taken directly from the existing real-duration-weighted inference result, with no pooling between performances. Failed windows leave timeline gaps. Failed recordings and recordings with no usable windows produce explanatory figures. PNG files mirror the input folder structure under `outputs/ver1/handpicked`; reruns replace report images, not source audio. A predicted raag here is one of the seven classes, not a separate family or thaat classification.

11 Verification and experiment status

Automated checks cover exact window boundaries; gain limits and silence; relative-pitch alignment for synthetic tones with different tonics; finite matrix dimensions; cache mismatch rejection; model dimensions/parameter counts; a CPU optimizer step and checkpoint reload; CPU fallback; native decoding, resampling, and FFmpeg fallback; short/long/silent/corrupt/tonic-failing inputs; duration aggregation; and notebook structure and code syntax.

An implementation smoke run prepares short real excerpts from all seven classes, uses the actual Essentia estimator, trains a single CPU epoch in a temporary run, and predicts from audio using that checkpoint. It verifies wiring rather than accuracy; synthetic-estimator unit tests are distinct from this real-audio check. The real-audio smoke check produced 14 usable windows without skips and completed one CPU training epoch. A post-extraction comparison with annotations found a median absolute tonic error of 500.51 cents, a median octave-corrected error of 10.72 cents, and nonzero nearest-octave displacement in five of 14 windows. Only four raw estimates were within 50 cents. This tiny correlated sample demonstrates a material tonic-alignment risk, not a general

accuracy estimate. No annotation-based correction was applied. Inspect broader tonic diagnostics before drawing conclusions from full model training.

The final verification results and extraction diagnostics are recorded in `docs/ver1_verification.md`. Full feature preparation, full training, GPU execution when unavailable, and external accuracy evaluation remain separate.

References

[1] J. Salamon, S. Gulati, and X. Serra. *A Multipitch Approach to Tonic Identification in Indian Classical Music*. ISMIR, 2012.

[2] Essentia. *TonicIndianArtMusic algorithm reference*. `https://essentia.upf.edu/reference/std_TonicIndianArtMusic.html`.

[3] Librosa. *STFT API reference*. `https://librosa.org/doc/main/api/generated/librosa.stft.html`.

[4] PyTorch. *Previous versions and CPU/CUDA installation*. `https://pytorch.org/get-started/previous-versions/`.